Module-2

1. Fraud Transaction Detection (Linear Search)

*** Problem Statement -**

A bank stores transaction IDs in the order they occur. Since transactions are not sorted, check whether a suspicious transaction ID exists.

*** Approach -**

→ Input n transaction IDs in array format.
→ Input target transaction ID inside the array.
→ Traverse array from first to last element.
→ From Traversing, target is found, so print "Fraud Transaction Found", Otherwise print "Transaction Not Found".

Code -

```
#include <iostream>
using namespace std;

int main() {
        int n;
        cout<<"Enter the size of array: ";
        cin >> n;
    int arr[n];
  cout << "Enter the elements of an array: ";
  for(int i=0 ; i< n ; i++) {
      cin >> arr[i];

  int target;
  cout << "Enter the target value: ";
    cin>> target;

  bool found= false;
```

```
for (int i=0; i<n; i++){
    if(arr[i]== target){
        found= true;
        break; }
    }
if(found){
   cout<< "Fraud Transaction Found";
} else{
    cout " Transaction not Found ";
}
 return 0;
}
```

* Time Complexity:- O(n)

2. Server Log Search (Binary Search)

* Problem Statement:-

A cloud company stores server log IDs in sorted order. Find whether a target log ID exists.

* Approach -

→ Input n server log IDs in array format with sorted order.
→ Input target server log ID in side the array.
→ Initialize low = 0 & high = n-1.
→ Find middle element.
→ If middle element equal to target, then stop.
→ If target is smaller, search in left half.
→ otherwise search in right half.
→ Repeat until target not found or low > high.

Code -

```
# include <iostream>
using namespace std;

int main() {
    int n;
 cout << " Enter the size of array: ";
  cin >> n;

  int arr[n];
 cout << " Enter the element of array : ";
  for(int i=0; i<n; i++)
      cin >> arr[i];

 int target;
 cout << " Enter the value of target : ";
   cin >> target;

int st = 0 , end = n-1;
 bool found = false;

while ( st <= end) {

 int mid = st + ( end-st) / 2 ;
  if (arr[mid] == target) {
       found = true;
       break;
  } else if ( target < arr[mid ]) {
        high = mid -1;
    } else {
        low = mid +1;
    }
    if (found) {
        cout << "Log found ";
    } else {
      cout << " Log not found ";
    } return 0; }
```

3. Sort Employee Salaries (Bubble Sort)

* Problem Statement -

Sort employee salaries in ascending order.

* Approach -

-> Compare adjacent elements.
-> Swap if left element is greater.
-> Repeat for all passes.
-> Largest element reaches the end after every pass.

* Code -

```
#include <iostream>
Using namespace std;

int main() {
    int n;
   cin>>n;

 int arr[n];
for(int i=0; i<n; i++){
      cin>> arr[i];

 for(int i=0; i<n-1; i++){
   for(int j=0; j<n-i-1; j++) {
       if(arr[j] > arr[j+1]) {
        swap(arr[j], arr[j+1]); }
      }
    }

  for(int i=0; i<n; i++){
     cout << arr[i] << " ";
     }
     return 0;
    }
```

4. Sort Website Response Time (Insertion sort)

* Problem Statement

Sort website response times in ascending order.

* Approach

→ Assume first element is sorted.
→ Pick next element as key.
→ Shift greater elements to the right.
→ Insert key at correct position.
→ Repeat until array becomes sorted.

* Code-

```
#include <iostream>;
using namespace std;

int main() {
    int n;
    cin >>n;
  int arr[n];
  for(int i=0 ; i<n; i++)
    cin >> arr[i];
 for(int i=1; i<n ; i++){
      int key = arr[i];
      int j = i-1;
 while(j>=0 && arr[j]>key){
      arr[j+1] = arr[j];
        j--;
    }
    arr[j+1] = key;
  }
    for(int i=0 ; i<n ;i++){
      cout << arr[i] << " ";
    }
    return 0;
  }
```

5. Prioritize Bug Reports (SelectionSort)

★ Problem Statement -

Sort bug priority scores in ascending order.

★ Approach -

→ Find minimum element.
→ Swap it with current position.
→ Repeat for remaining array.
→ Continue until array becomes sorted.

★ Code -

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
  int arr[n];
 for(int i=0; i<n; i++)
    cin >> arr[i];
 for(int i=0; i<n-1; i++){
      int minIndex = i;
  for(int j=i+1; j<n; j++){
    if(arr[j] < arr[minIndex])
      minIndex=j;
   }
   swap(arr[i], arr[minIndex]);
  }
   for(int i=0; i<n; i++){
     cout << arr[i] << " ";
     }
   return 0;
   }
```

1. Fraud Transaction Detection (Linear Search)

Problem Statement

A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists.

Write a program using Linear Search.

Input Format

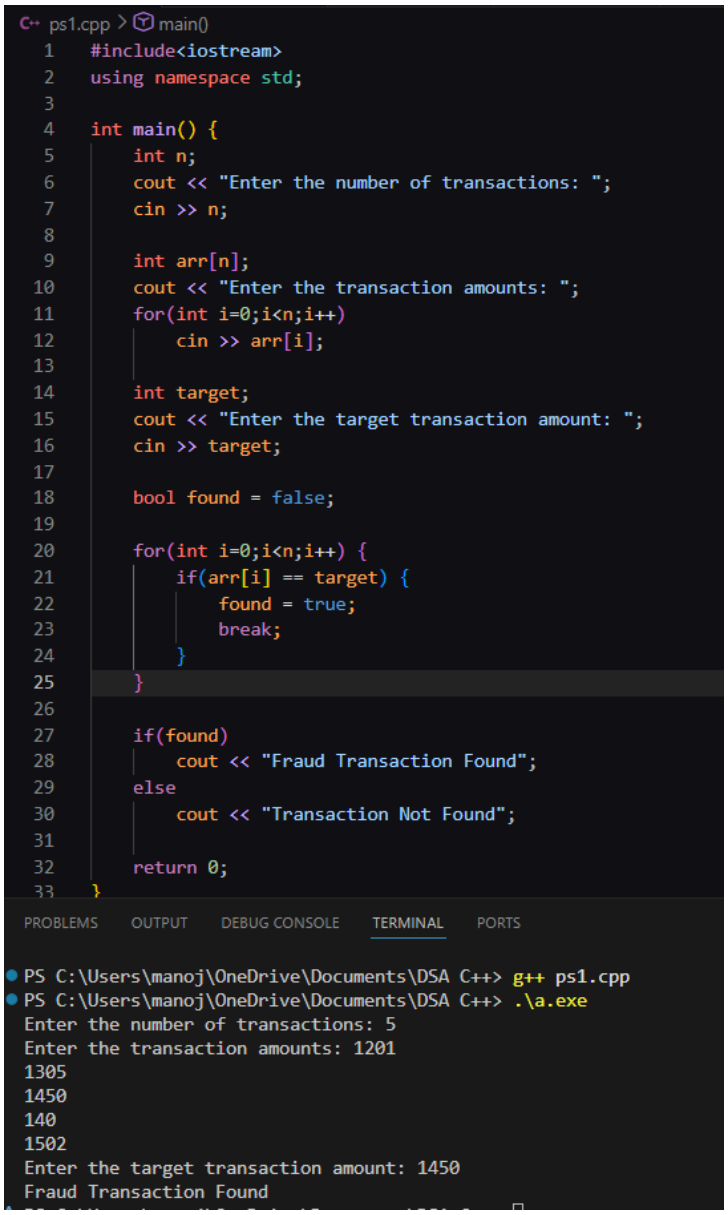
• First line contains integer N

• Second line contains N transaction IDs

• Third line contains suspicious transaction ID

Output Format

Print: Fraud Transaction Found or Transaction Not Found



2. Server Log Search (Binary Search)

Problem Statement

A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists. Write a program using Binary Search.

Input Format

• First line contains N

• Second line contains sorted log IDs

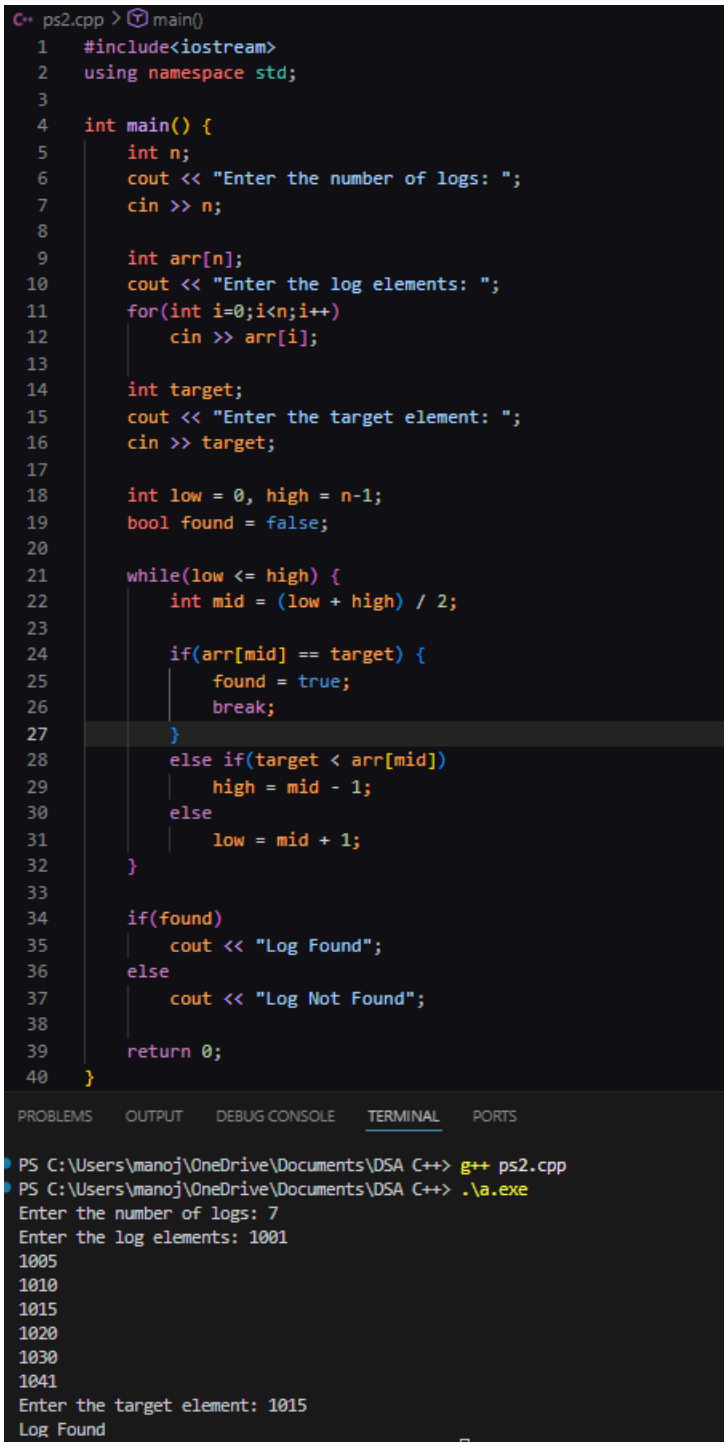
• Third line contains target log ID

Output Format

Print: Log Found or Log Not Found

Constraints

• 1 ≤ N ≤ 10^5



```cpp
#include<iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter the number of logs: ";
    cin >> n;

    int arr[n];
    cout << "Enter the log elements: ";
    for(int i=0;i<n;i++)
        cin >> arr[i];

    int target;
    cout << "Enter the target element: ";
    cin >> target;

    int low = 0, high = n-1;
    bool found = false;

    while(low <= high) {
        int mid = (low + high) / 2;

        if(arr[mid] == target) {
            found = true;
            break;
        }
        else if(target < arr[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }

    if(found)
        cout << "Log Found";
    else
        cout << "Log Not Found";

    return 0;
}
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps2.cpp
PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
Enter the number of logs: 7
Enter the log elements: 1001
1005
1010
1015
1020
1030
1041
Enter the target element: 1015
Log Found
```

3. Sort Employee Salaries (Bubble Sort)

Problem Statement

A company wants to sort employee salaries in ascending order for payroll analysis. Write a program using Bubble Sort.

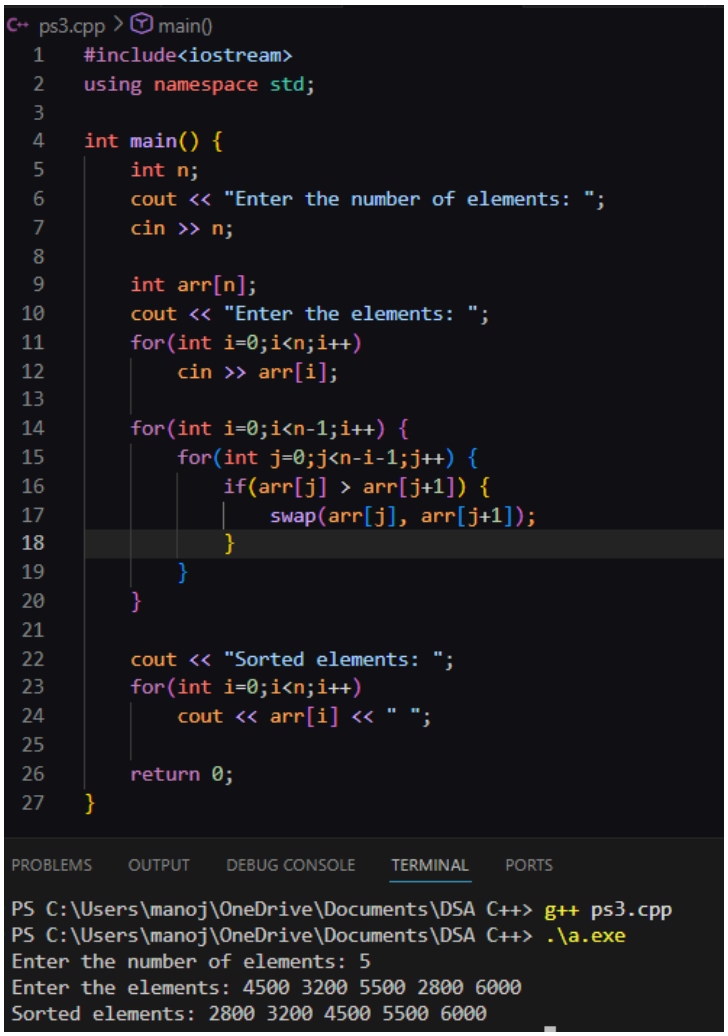
Input Format

• First line contains N

• Second line contains salaries

Output Format

Print sorted salaries.



```cpp
#include<iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter the elements: ";
    for(int i=0;i<n;i++)
        cin >> arr[i];

    for(int i=0;i<n-1;i++) {
        for(int j=0;j<n-i-1;j++) {
            if(arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }

    cout << "Sorted elements: ";
    for(int i=0;i<n;i++)
        cout << arr[i] << " ";

    return 0;
}
```

```
PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps3.cpp
PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
Enter the number of elements: 5
Enter the elements: 4500 3200 5500 2800 6000
Sorted elements: 2800 3200 4500 5500 6000
```

4. Sort Website Response Time (Insertion Sort)

Problem Statement

A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position. Write a program using Insertion Sort.

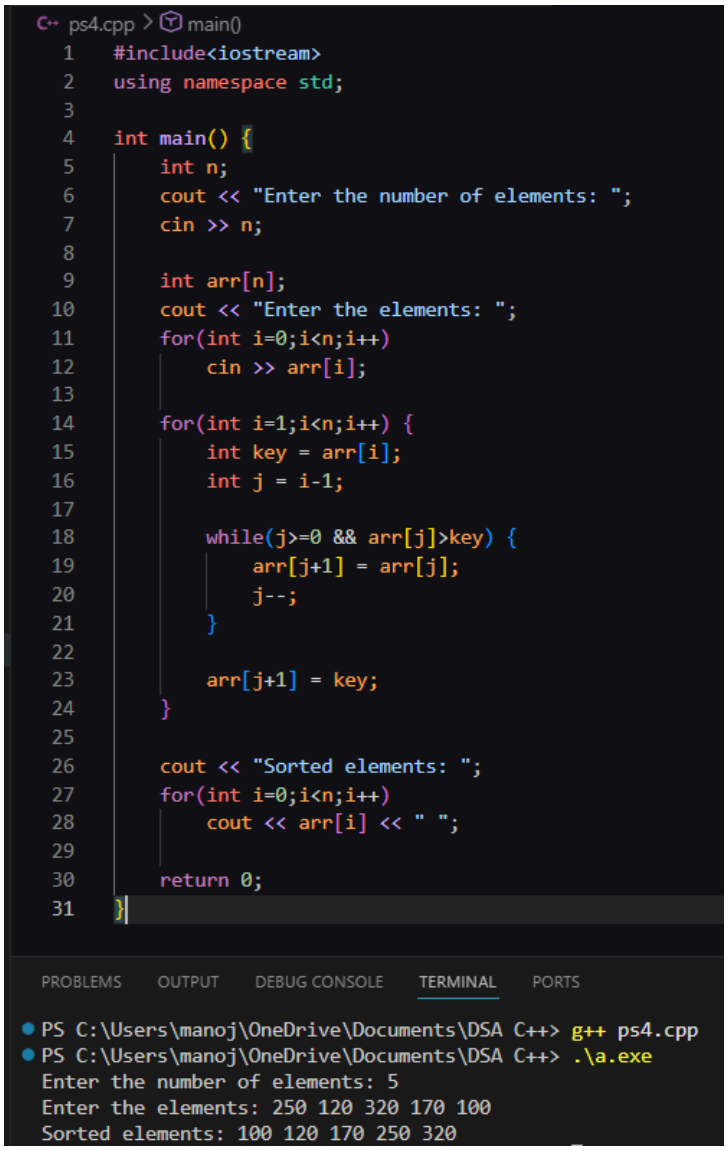
Input Format

• First line contains N

• Second line contains response times

Output Format

Print sorted response times.



5. Prioritize Bug Reports (Selection Sort)

Problem Statement

A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order. Write a program using Selection Sort.

Input Format

• First line contains N

• Second line contains priority scores

Output Format

Print sorted priority scores.

ps5.cpp > main()

```cpp
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter the elements: ";
    for(int i=0;i<n;i++)
        cin >> arr[i];

    for(int i=0;i<n-1;i++) {

        int minIndex = i;

        for(int j=i+1;j<n;j++) {
            if(arr[j] < arr[minIndex])
                minIndex = j;
        }

        swap(arr[i], arr[minIndex]);
    }
    cout << "Sorted elements: ";
    for(int i=0;i<n;i++)
        cout << arr[i] << " ";

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps5.cpp
PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
Enter the number of elements: 5
Enter the elements: 9 2 7 1 5
Sorted elements: 1 2 5 7 9
```